

# ➤ Programación III

Plataforma de Streaming

## Integrantes:

- Alvarez Lovera Sandra Sofia
- Castilla Escudero Marcelo Daniel
- Cuadros Villegas Denilson Jermai
- Dávila Bazán Santiago Sebastián
- Torres Chafloque Enrique Eusebio



# Contenido

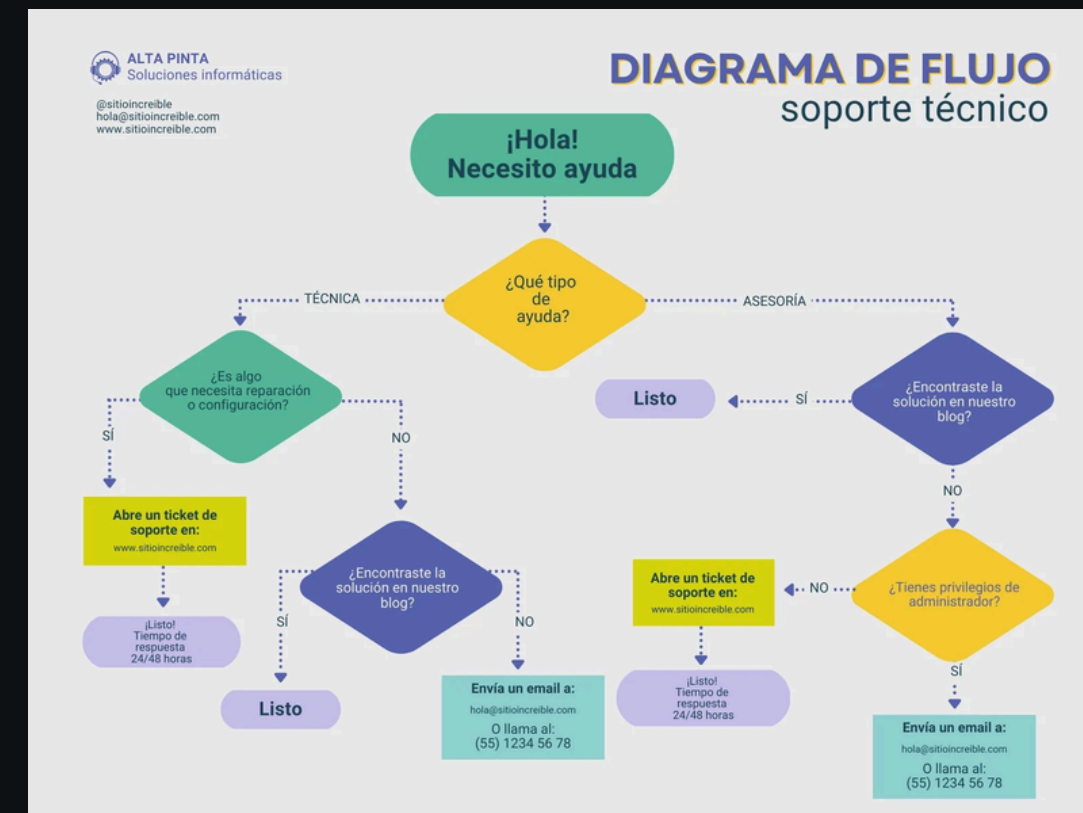
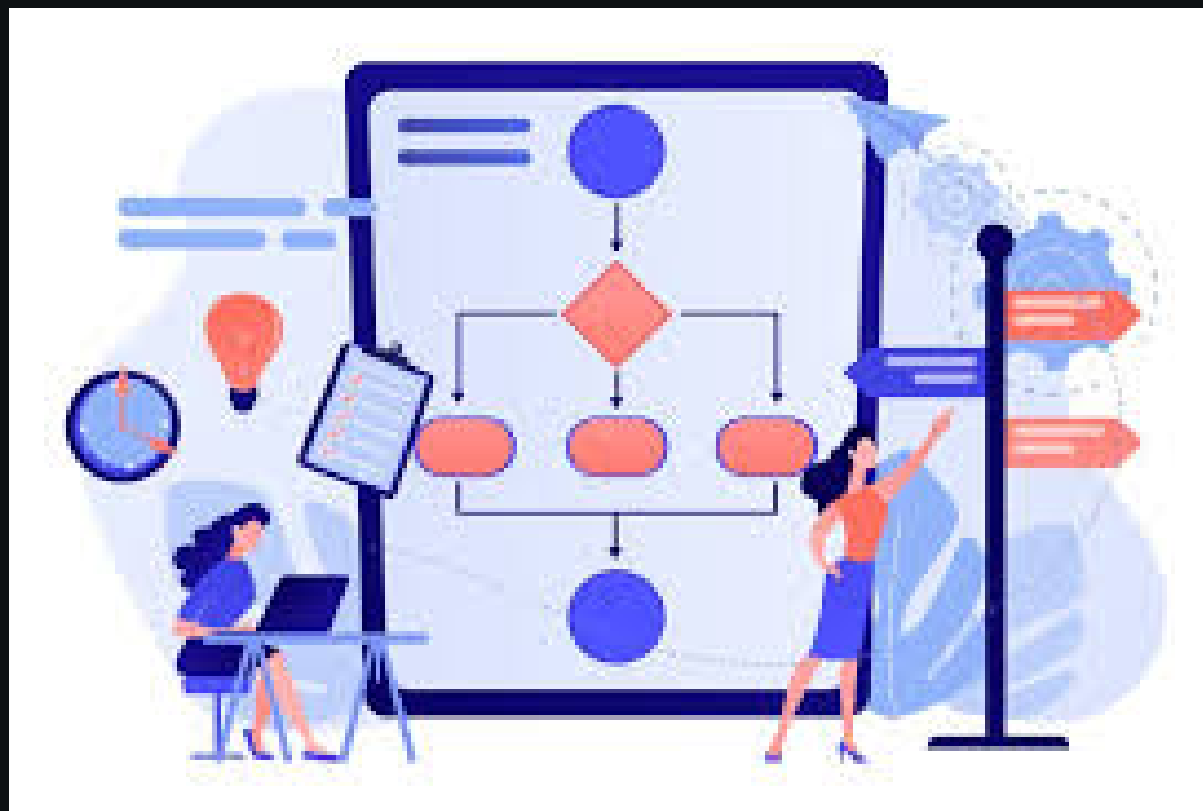
- 01. Introducción
- 02. Preprocesamiento de datos
- 03. Árbol elegido
- 04. Búsqueda
- 05. Núcleo del sistema (interfaz)



# Introducción

main

- develop
- feature/trie-core ← Trie + searchByPrefix + DFS
- feature/ngram-index ← NgramIndex para substring
- feature/core-clases ← Pelicula, Usuario, Sesion, etc.
- feature/csv-loader ← Lectura y normalización del CSV
- feature/ranking ← Algoritmo de importancia

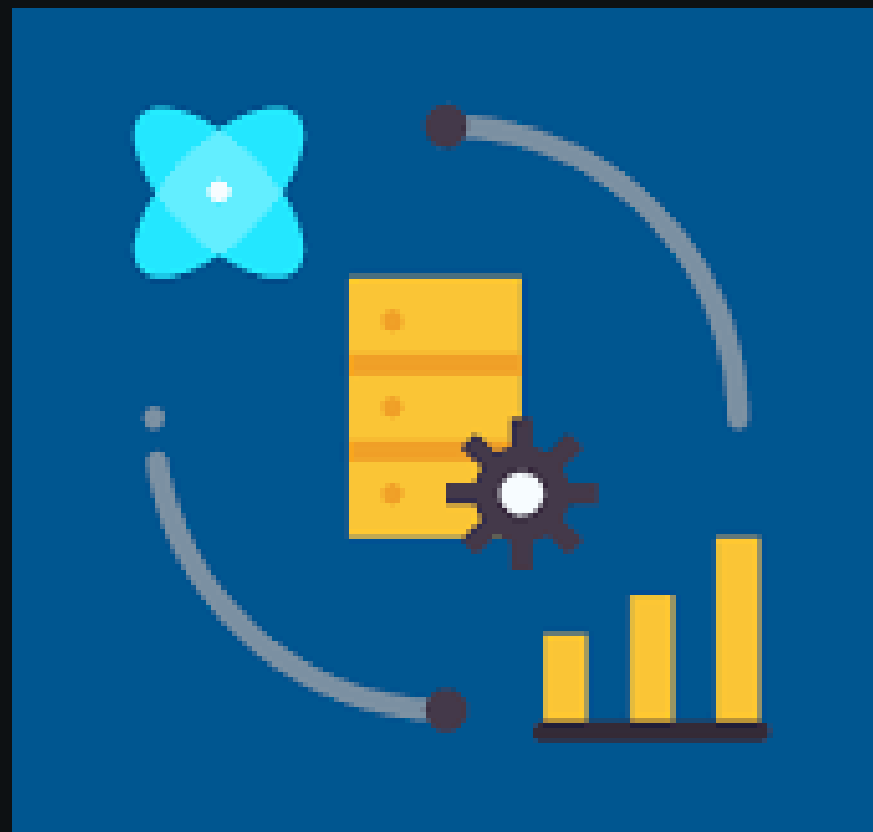


# CARGA Y LIMPIEZA DEL DATAFRAME



- Leer el archivo csv que contiene a las películas
- Separar cada línea en columnas
- Crear objetos Movie para almacenar cada película
- Limpiar datos (eliminar espacios sobrantes, convertir textos a minúsculas, eliminar valores desconocidos, normalizar separadores)
- Convertir actores y géneros en vectores para manejar listas de datos
- Reemplazar valores desconocidos por strings vacío
- Elimina película inválidas
- Evita duplicados según título, año y origen
- Guarda todas las películas válidas en un dataframe `vector<Movie*>`
- Libera memoria para evitar fugas

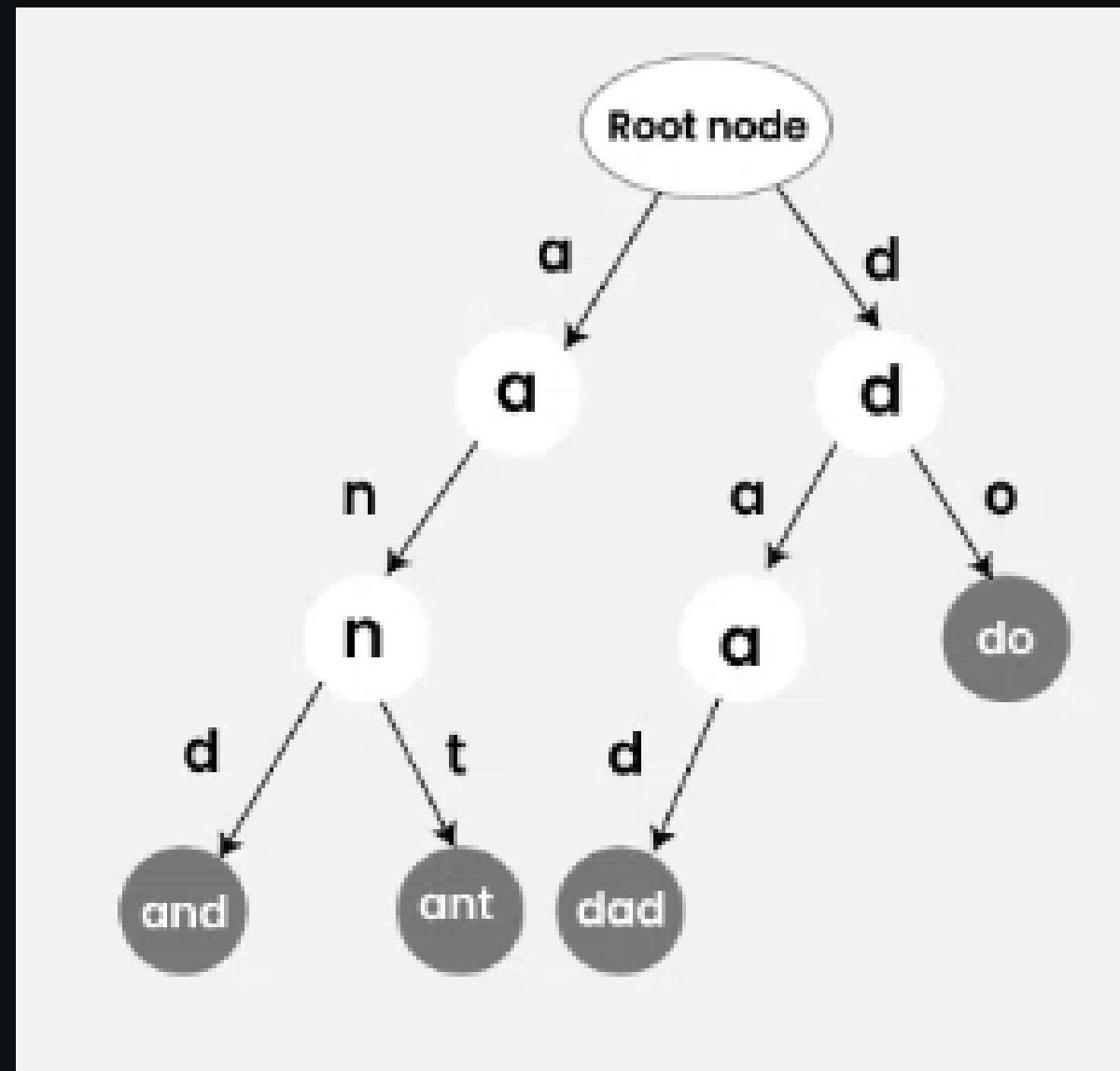
# PREPROCESAMIENTO PARA EL TRIE



- Cargar diccionario de palabras despreciables desde un archivo txt
- Tokenizar títulos y plots, separar en palabras individuales, eliminar puntuación y convertir a minúsculas
- Filtrar tokens menores a 2 caracteres y palabras despreciables.
- Generar n-gramas de tamaño 3 por cada token para permitir búsqueda por fragmentos
- Construir vocabulario con las 91,980 palabras únicas del dataset como base para el Trie
- Liberar todos los objetos Movie\* del dataframe al finalizar



# Árbol elegido: **TRIE**



## **Ventajas**

Búsqueda  $O(L)$  – igual con 10 o 1 millón de películas.  $L$  = largo de la palabra buscada. Excelente para autocompletar y prefijos.

## **Desventajas**

Consume más memoria que un hash si el diccionario es muy grande. No es óptimo para búsqueda por substring puro (por eso complementamos con N-gramas).

# Complejidad Algorítmica

Inserción

$O(L)$

$L$  = largo de la palabra. Se recorren  $L$  nodos, uno por carácter.

Búsqueda exacta

$O(L)$

Se baja por el árbol carácter a carácter hasta encontrar el nodo final.

Búsqueda prefijo

$O(L)$

Igual que búsqueda exacta pero sin requerir `is_end_of_word`.

Espacio

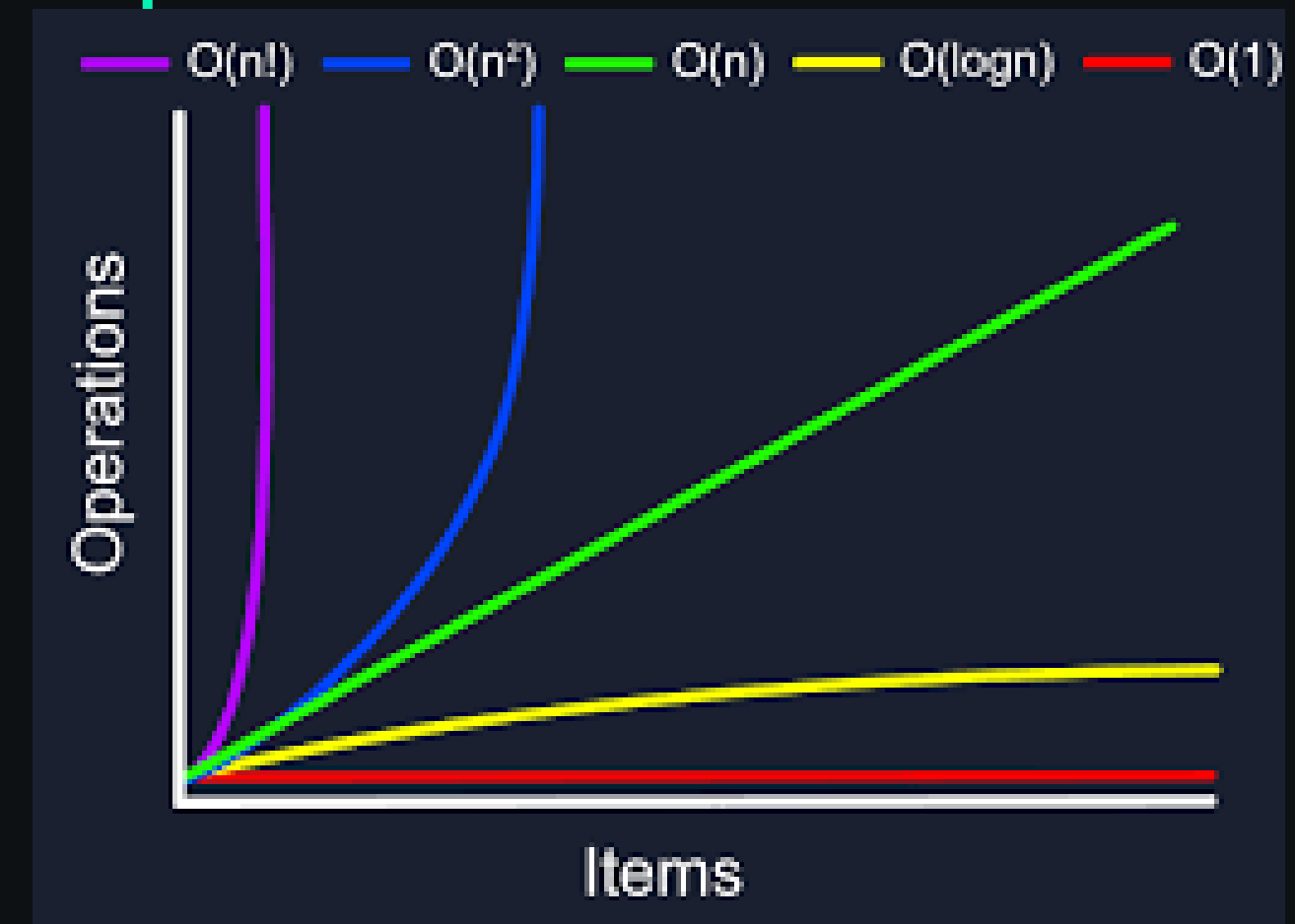
$O(N \cdot L)$

$N$  = número de palabras,  $L$  = largo promedio. Cada carácter ocupa un nodo.

N-grama substring

$O(L \cdot k)$

$k$  = tamaño del n-grama (ej: trigramas). Permite "bar" → "barco".



# Pseudocódigo del trie

```
FUNCIÓN cargarCatalogo(archivo_csv, trie, catalogo):  
    PARA CADA linea EN leer_csv(archivo_csv):  
        pelicula = parsear(linea)  
        catalogo.agregar(pelicula)  
  
        tokens = tokenizar(pelicula.titulo + " " + pelicula.sinopsis)  
        PARA CADA token EN tokens:  
            token = normalizar(token) // lowercase, sin tildes  
            SI token NO es stopword:  
                trie.insertar(token, pelicula.id)  
  
            // N-grama para substring: "barco" → "bar", "arc", "rco"  
            PARA i=0 MIENTRAS i+3 ≤ len(token):  
                ngrama = token[i..i+3]  
                ngram_index.insertar(ngrama, pelicula.id)  
                i++
```

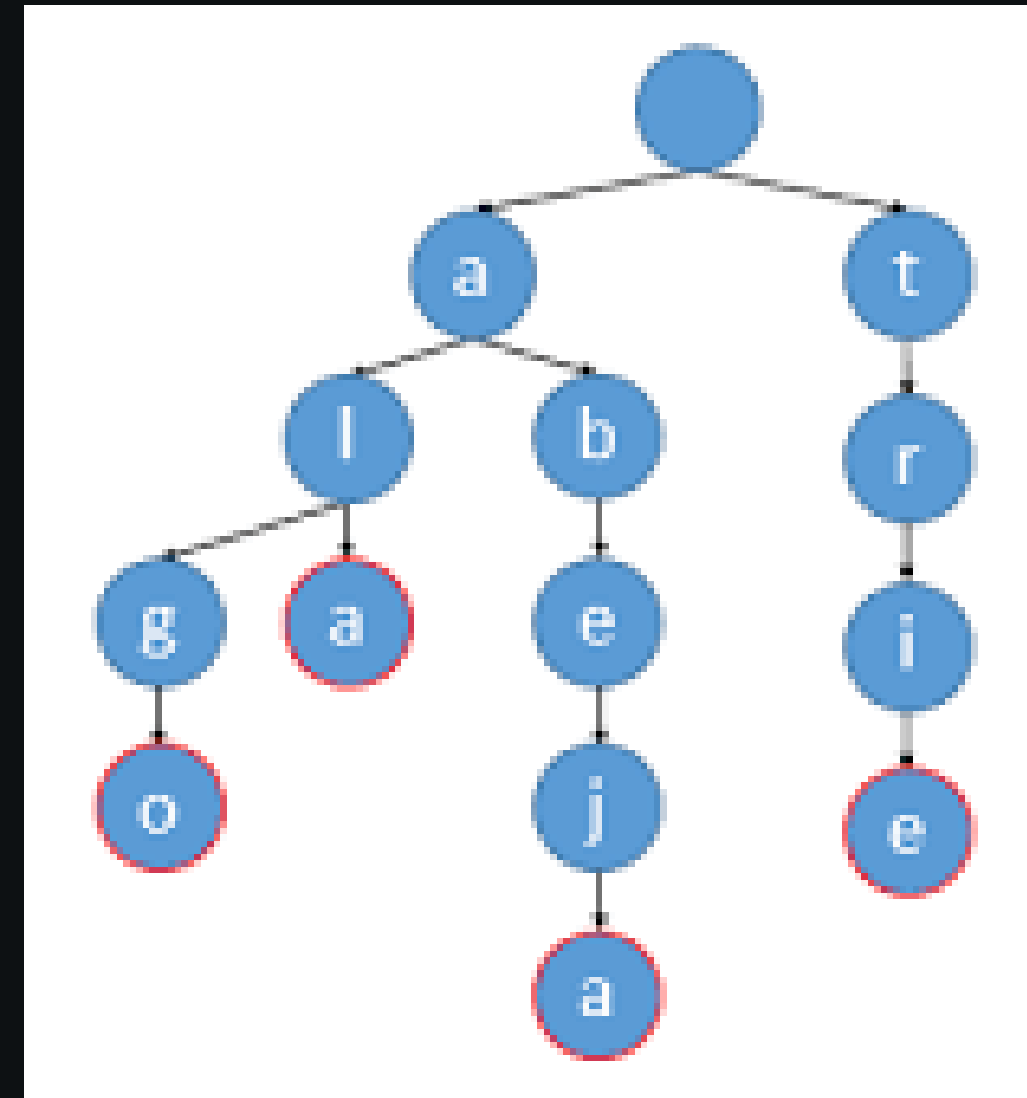
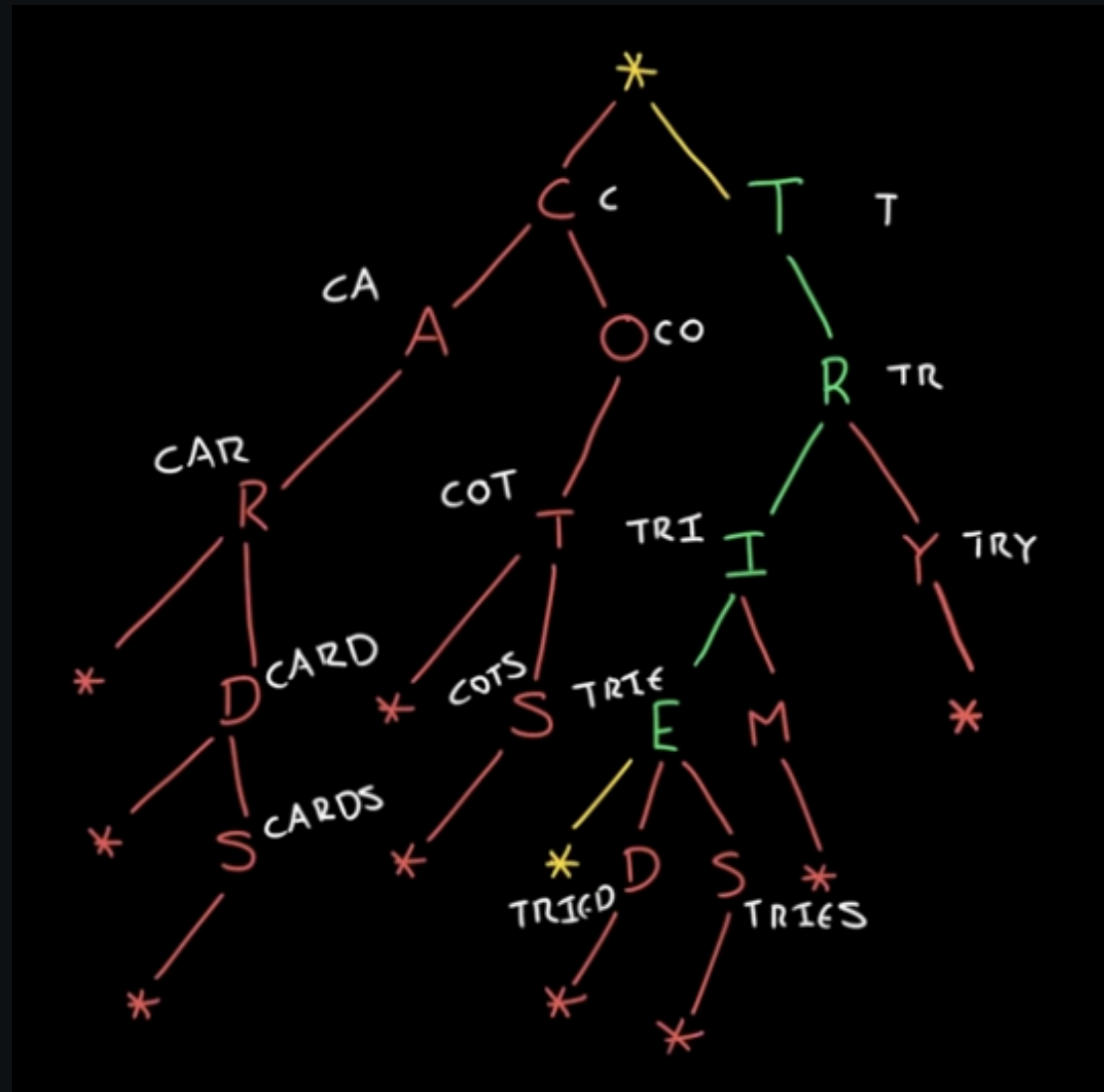
....



# Pseudocódigo del trie

```
...  
// Insertar tags especiales  
trie.insertar("director:" + normalizar(pelicula.director), pelicula.id)
```

```
PARA CADA genero EN pelicula.generos:  
trie.insertar("tag:" + normalizar(genero), pelicula.id)
```



# Pseudocódigo del trie

```
FUNCIÓN insertar(trie, palabra, movie_id):  
  
    // Validar entrada  
    SI palabra == "" O palabra == nulo:  
        RETORNAR  
  
    nodo_actual = raiz  
    i = 0  
  
    PARA i=0 MIENTRAS i < len(palabra):  
        caracter = palabra[i]  
  
        SI caracter NO existe en nodo_actual.hijos:  
            nodo_actual.hijos[caracter] = nuevo_nodo()  
  
        nodo_actual = nodo_actual.hijos[caracter]  
        i++  
  
    nodo_actual.es_fin = verdadero  
    ....
```

# Pseudocódigo del trie

```
SI movie_id NO está en nodo_actual.movie_ids:  
    nodo_actual.movie_ids.agregar(movie_id)  
FUNCIÓN insertar(trie, palabra, movie_id):  
    nodo_actual = raiz  
  
    PARA CADA caracter EN palabra:  
        SI caracter NO existe en nodo_actual.hijos:  
            nodo_actual.hijos[caracter] = nuevo_nodo()  
            nodo_actual = nodo_actual.hijos[caracter]  
  
    nodo_actual.es_fin = verdadero  
  
    SI movie_id NO está en nodo_actual.movie_ids:  
        nodo_actual.movie_ids.agregar(movie_id)
```

# Pseudocódigo de la búsqueda

```
FUNCIÓN buscar(consulta, trie, ngram_index) → lista_ids:
```

```
    consulta = normalizar(consulta)
```

```
    palabras = tokenizar(consulta)
```

```
    ids_acumulados = {}
```

```
    PARA CADA palabra EN palabras:
```

```
        // Caso 1: tag especial → "tag:horror",  
        "director:nolan"
```

```
        SI palabra contiene ":":
```

```
            ids = trie.buscar_exacto(palabra)
```

```
        // Caso 2: palabra de 3 letras o más
```

```
        SINO SI len(palabra) >= 3:
```

```
            ids = trie.buscar_exacto(palabra)
```

```
            ids += trie.buscar_por_prefijo(palabra)
```

```
            ids += ngram_index.buscar(palabra)
```

```
        // Caso 3: palabra de 2 letras → "it", "up", "us"
```

```
        SINO SI len(palabra) == 2:
```

```
            ids = ngram_index.buscar(palabra)
```

```
        // Caso 4: 1 letra → ignorar
```

```
        SINO:
```

```
            CONTINUAR
```

```
    ids_acumulados.unir(ids)
```

```
    RETORNAR rankear(ids_acumulados, palabras)
```



# Pseudocódigo de la búsqueda

```
FUNCIÓN buscar_por_prefijo(trie, prefijo) → lista_ids:  
    nodo = raíz  
    PARA CADA caracter EN prefijo:  
        SI caracter NO en nodo.hijos: RETORNAR []  
        nodo = nodo.hijos[caracter]  
  
    // DFS desde aquí para recoger todos los IDs del  
    subárbol  
    RETORNAR dfs_recoger_ids(nodo)
```





# Pseudocódigo de ranking

```
FUNCIÓN rankear(ids, palabras_buscadas, usuario_activo):  
    scores = {}
```

```
    PARA CADA id EN ids:  
        pelicula = catalogo.getPelicula(id)  
        pts = 0
```

```
        PARA CADA palabra EN palabras_buscadas:  
            SI palabra EN pelicula.titulo:           pts += 3  
            SI palabra EN pelicula.director:         pts += 2  
            SI palabra EN pelicula.sinopsis:          pts += 1
```

```
        SI pelicula comparte_genero_con likes_usuario: pts += 2  
        pts = pts * (pelicula.score / 10.0)  
        scores[id] = pts
```

```
    ORDENAR ids POR scores[id] DESCENDENTE  
    RETORNAR ids // mostrar de 5 en 5
```



# Sobre el avance de la interfaz

Se busco simular la experiencia entre el usuario y el proyecto final con un codigo consistente y que no se rompa

- Se uso recursividad, variables y condiciones, imprimir una guía clara para el usuario de todo el sistema, y opciones para terminar el proyecto si es que así lo desea el usuario
- NO se aplico la logica final para el avance de la interfaz. En su reemplazo, se usaron variables genéricas temporales que imitan el comportamiento de lo que serán las variables finales del proyecto. Strings fijos, listas vacias, etc.
- Se busco orientar mediante mensajes adicionales sobre el rumbo o el plan que se tiene para cada parte de la interfaz y el codigo en general, esto para demostrar que ya se tiene el desarrollo en mente para todas partes de esta demo del interfaz y del proyecto.

